AgentAuth 1.0 - Complete API Documentation

RFC-0111/0115 Compliant Authorization Framework

AgentAuth Community

November 17, 2025

Contents

Chapter 1

AgentAuth+ API Usage Examples

Complete examples for using the AgentAuth+ admin API endpoints.

1.1 Audit Export Examples

1.1.1 Export Last 24 Hours (JSON)

```
curl -X POST http://localhost:8080/api/admin/audit/export \
  -H "Content-Type: application/json" \
  -d '{
    "format": "json",
    "dateRange": "last-24h",
    "compressed": false
  }'

# Response:
{
  "jobId": "uuid-here",
  "status": "pending",
  "format": "json",
  "createdAt": "2025-12-29T00:00:00Z",
  "expiresAt": "2025-12-30T00:00:00Z"
}
```

1.1.2 Export Last Week (CSV, Compressed)

```
curl -X POST http://localhost:8080/api/admin/audit/export \
  -H "Content-Type: application/json" \
  -d '{
    "format": "csv",
    "dateRange": "last-7d",
    "compressed": true,
    "category": "security",
```

```
    "severity": "high"
  }'
```

1.1.3 Check Export Status

```
JOB_ID="your-job-id-here"
curl http://localhost:8080/api/admin/audit/export/$JOB_ID | jq .

# Response when complete:
{
  "jobId": "...",
  "status": "completed",
  "format": "json",
  "fileSize": 1024,
  "totalEvents": 150,
  "completedAt": "2025-12-29T00:05:00Z"
}
```

1.1.4 Download Completed Export

```
JOB_ID="your-job-id-here"
curl -o audit-export.json \
  http://localhost:8080/api/admin/audit/export/$JOB_ID/download
```

1.1.5 Delete Export Job

```
curl -X DELETE http://localhost:8080/api/admin/audit/export/$JOB_ID
```

1.2 API Key Management Examples

1.2.1 Create API Key

```
curl -X POST http://localhost:8080/api/admin/api-keys \
  -H "Content-Type: application/json" \
  -d '{
    "tenantId": "acme-corp",
    "keyName": "Production Service",
    "description": "Main backend service API key",
    "scopes": ["poa:read", "poa:write", "delegation:create"],
    "rateLimitPerMinute": 100,
    "rateLimitPerHour": 5000,
    "createdBy": "admin@acme.com"
  }'

# Response (secretKey only shown once!):
{
  "apiKey": {
```

```
    "id": "key-123",
    "keyId": "agentauth_sk_...",
    "name": "Production Service",
    "scopes": ["poa:read", "poa:write"],
    "createdAt": "2025-12-29T00:00:00Z"
  },
  "secretKey": "agentauth_sk_live_abc123...",
  "message": "Save this key securely - it won't be shown again"
}
```

1.2.2 List API Keys

```
curl "http://localhost:8080/api/admin/api-keys?tenant_id=acme-corp" | jq .
```

1.2.3 Get API Key Details

```
curl http://localhost:8080/api/admin/api-keys/key-123 | jq .
```

1.2.4 Get API Key Usage Statistics

```
curl "http://localhost:8080/api/admin/api-keys/key-123/usage?tenant_id=acme-corp" | jq .

# Response:
{
  "totalRequests": 1500,
  "successRate": 98.5,
  "avgResponseTime": 45,
  "requestsByEndpoint": {
    "/api/v1/poa": 800,
    "/api/v1/delegation": 700
  }
}
```

1.2.5 Revoke API Key

```
curl -X POST http://localhost:8080/api/admin/api-keys/key-123/revoke \
  -H "Content-Type: application/json" \
  -d '{"tenant_id": "acme-corp"}'
```

1.3 Audit Events Examples

1.3.1 List Recent Events

```
curl "http://localhost:8080/api/admin/audit/events?limit=10&offset=0" | jq .
```

1.3.2 Filter by Category

```
curl "http://localhost:8080/api/admin/audit/events?category=security&severity=high" | jq .
```

1.3.3 Filter by User and Action

```
curl "http://localhost:8080/api/admin/audit/events?actor=user@example.com&action=poa_created"
```

1.4 Compliance Reports Examples

1.4.1 Get All Compliance Reports

```
curl http://localhost:8080/api/admin/audit/compliance | jq .

# Response:
{
  "reports": [
    {
      "id": "report-1",
      "framework": "GDPR",
      "standard": "General Data Protection Regulation",
      "status": "compliant",
      "coverage": 95,
      "violations": 2,
      "lastAudit": "2025-12-29T00:00:00Z"
    }
  ]
}
```

1.5 Metrics Examples

1.5.1 Get Audit Metrics

```
curl http://localhost:8080/api/admin/audit/metrics | jq .

# Response:
{
  "total_events": 5000,
  "events_by_category": {
    "auth": 1500,
    "admin": 800,
    "system": 2700
  },
  "events_by_severity": {
    "critical": 5,
    "high": 50,
    "medium": 500,
    "low": 4445
  },
```

```
  "siem_integrations": {
    "total": 2,
    "active": 2,
    "events_sent": 4950
  }
}
```

1.6 Complete Workflow Example

1.6.1 Complete Audit Export Workflow

```bash
#!/bin/bash

# 1. Create export job
echo "Creating export job..."
RESPONSE=$(curl -s -X POST http://localhost:8080/api/admin/audit/export \
  -H "Content-Type: application/json" \
  -d '{
    "format": "json",
    "dateRange": "last-7d",
    "compressed": false
  }')

JOB_ID=$(echo $RESPONSE | jq -r '.jobId')
echo "Job ID: $JOB_ID"

# 2. Wait for completion
echo "Waiting for export to complete..."
while true; do
  STATUS=$(curl -s http://localhost:8080/api/admin/audit/export/$JOB_ID | jq -r '.status')
  echo "Status: $STATUS"

  if [ "$STATUS" = "completed" ]; then
    break
  fi

  sleep 2
done

# 3. Download export
echo "Downloading export..."
curl -o audit-export-$(date +%Y%m%d).json \
  http://localhost:8080/api/admin/audit/export/$JOB_ID/download

echo "Export complete!"
```

```
# 4. Optional: Delete job
curl -X DELETE http://localhost:8080/api/admin/audit/export/$JOB_ID
```

1.7 Python Examples

1.7.1 Python: Export Audit Logs

```python
import requests
import time
import json

BASE_URL = "http://localhost:8080"

# Create export job
response = requests.post(
    f"{BASE_URL}/api/admin/audit/export",
    json={
        "format": "json",
        "dateRange": "last-24h",
        "compressed": False
    }
)
job_id = response.json()["jobId"]
print(f"Job ID: {job_id}")

# Wait for completion
while True:
    status_response = requests.get(
        f"{BASE_URL}/api/admin/audit/export/{job_id}"
    )
    status = status_response.json()["status"]

    if status == "completed":
        break

    time.sleep(2)

# Download export
download_response = requests.get(
    f"{BASE_URL}/api/admin/audit/export/{job_id}/download"
)

with open("audit-export.json", "wb") as f:
    f.write(download_response.content)

print("Export downloaded successfully")
```

1.7.2 Python: Create API Key

```python
import requests

response = requests.post(
    "http://localhost:8080/api/admin/api-keys",
    json={
        "tenantId": "my-tenant",
        "keyName": "Python Service",
        "description": "API key for Python application",
        "scopes": ["poa:read"],
        "rateLimitPerMinute": 60,
        "createdBy": "admin"
    }
)

result = response.json()
print(f"API Key ID: {result['apiKey']['id']}")
print(f"Secret Key: {result['secretKey']}")
print("SAVE THIS KEY - it won't be shown again!")
```

1.8 Revocation Transparency Endpoint Examples (Beta)

For internal model details (Merkle accumulator, Signed Tree Heads v1/v2, multi-sig threshold & weights, persistence, consistency proofs) consult `REVOCATION_TRANSPARENCY.md`. ### Inclusion Proof `GET /api/v1/token/revocation/proof` (exactly one of `id`, `index`, or `hash` required) Example:

```
curl -s "http://localhost:8080/api/v1/token/revocation/proof?hash=4f8c..." | jq
```

Success response:

```
{
    "root": "b6f1c9e3dc...",
    "leaf_hash": "4f8c2e91a5...",
    "leaf_digest": "0f9a7c...",
    "index": 41,
    "chain_length": 42,
    "proof": [
    {"sibling": "ab12...", "position": "R"},
    {"sibling": "77fe...", "position": "L"}
    ],
    "verified": true
}
```

Error (missing query parameter):

```
{
    "error": "missing_query_param",
    "message": "one of id, index, hash must be provided"
```

```
}
```

1.8.1 Consistency Proof (Prototype)

`GET /api/v1/token/revocation/consistency?start=<start_index>` Example:

```
curl -s "http://localhost:8080/api/v1/token/revocation/consistency?start=0" | jq
```

Success response:

```
{
    "start_length": 5,
    "end_length": 42,
    "start_root": "91ab...",
    "end_root": "b6f1c9...",
    "new_leaves": ["4f8c2e91a5...", "8aa7bc11e3..."],
    "verified": true
}
```

1.8.2 Revocation Consistency Proof V2 (Logarithmic)

Endpoint: `GET /api/v1/token/revocation/consistency_v2?start=<tree_head_index>`

Returns an RFC6962-style (prototype) JSON object:

```
{
    "success": true,
    "proof": {
        "start_length": <int>,
        "end_length": <int>,
        "path": ["<hexNode>", ...],
        "start_root": "<hex>",
        "end_root": "<hex>"
    },
    "latest_tree_head": { ... SignedTreeHead ... }
}
```

Characteristics: - Path size is logarithmic relative to end_length. - Demonstrates append-only growth from `start_length` to `end_length`. - Provides optional prefix subtree decomposition (`prefix_roots`, `prefix_sizes`) representing maximal power-of-two aligned blocks covering the first `start_length` leaves (binary decomposition). Sum(prefix_sizes) == start_length. - If the revocation chain is currently empty (no events appended), `latest_tree_head` MAY be `null` (or omitted) because `SignTreeHead()` now returns `nil` for empty chains (see "Empty Chain Behavior" in `REVOCATION_TRANSPARENCY.md`). Clients must handle this case gracefully. - Current verification (`VerifyConsistencyProofV2`) still rebuilds the start tree for canonical root validation, but additionally validates each prefix root matches the corresponding subtree. Experimental fast reconstruction (feature flag `AGENTAUTH_CONSISTENCY_V2_FAST=1`) currently succeeds only when `start_length` is a power of two (single prefix block); multi-block reconstruction is deferred.

Additional Fields (Optimization Phase): | Field | Description | |——-|————-| | `prefix_bridges` | Sequence of intermediate merged subtree digests produced by right-to-left reduction over the prefix

blocks; enables fast deterministic reconstruction of `start_root` without full tree rebuild when fast flag enabled. |

Generation Algorithm (Optimized): 1. Streaming Segment Stack: Leaves up to `start_length` are ingested one by one, forming a stack of `(hash,size)` segments. Adjacent equal-sized segments merge immediately (size doubles) yielding a canonical power-of-two decomposition of the processed prefix. 2. Prefix Blocks: The final stack embodies `prefix_roots`/`prefix_sizes` (left → right). This avoids constructing all tree levels (`O(n)` memory) while retaining `O(n)` leaf hashing cost. 3. Bridges Construction: A right-to-left reduction merges the last two segments repeatedly (with consolidation when equal sizes arise) producing `prefix_bridges`. These bridges mirror the exact merge order of the canonical tree formation for heterogeneous block boundaries. 4. Consistency Path: (Current phase) A temporary Merkle tree is still built to derive the logarithmic path between historical and latest tree heads. Future phase will replace this with pure streaming interval math eliminating remaining `O(n)` generation overhead.

Fast Reconstruction: `ReconstructStartRootFromPrefixBlocks(prefix_roots, prefix_sizes, start_length, prefix_bridges)` validates invariants and performs a reduction using bridges to obtain `start_root` in `O(k)` where `k = len(prefix_roots) + len(prefix_bridges)` (typically `< 2*log2(start_length)`). If the feature flag `AGENTAUTH_CONSISTENCY_V2_FAST=1` is set, verification attempts this reconstruction and compares the result against the canonical rebuilt root; any mismatch causes verification failure.

Auditor Endpoint Enhancement: `GET /api/v1/token/revocation/audit_consistency?start=<tree_head_ind` returns both legacy (full rebuild) and fast reconstruction timings:

```
{
    "success": true,
    "proof": { ... },
    "legacy_duration_ns": 1203487,
    "fast_duration_ns": 19834,
    "fast_root_matches": true
}
```

Use this to externally validate performance characteristics and detect any divergence between approaches.

Benchmark Snapshot (Generation Phase): | Leaves | New Gen (ns/op) | Legacy Gen (ns/op) | Alloc (new) | Alloc (legacy) | |——–|——————|——————|————-|——————-| | 64 | ~32.5µs | ~51.8µs | 73KB | 118KB | | 256 | ~129.7µs | ~203.7µs | 294KB | 472KB | | 1024 | ~530.8µs | ~837.3µs | 1.20MB | 1.93MB | | 4096 | ~2.22ms | ~3.55ms | 4.77MB | 7.70MB | (*Apple M3 Pro, Go 1.22+, benchtime 0.2s; values approximate; see `consistency_generation_bench_test.go` for details.)

Client Guidance (Updated): 1. Validate prefix decomposition (`prefix_roots`/`prefix_sizes`) and optionally `prefix_bridges` if using fast path. 2. Attempt fast reconstruction when flag enabled; fall back to canonical full rebuild if disabled or reconstruction returns empty string. 3. Verify append-only via path as before; ensure `path` length within logarithmic bound. 4. Record auditor metrics for monitoring (e.g., anomaly detection if fast_root mismatch appears).

Limitations (Updated): - Consistency path still incurs an `O(n)` temporary tree build (optimization forthcoming). - Range hashing and interval-based sibling derivation are deferred to next phase to eliminate remaining rebuild cost. - Prefix bridges rely on deterministic merge ordering; any alteration in merge policy requires bridge sequence regeneration logic update.

Client Guidance: 1. Verify `start_root` and `end_root` against locally reconstructed Merkle trees (or incremental state). 2. Ensure `path` length is within allowed logarithmic bounds. 3. (Future) Reconstruct start root using path-only algorithm to avoid full rebuild.

Use Cases: - Efficient audit of revocation chain evolution. - Potential integration with external transparency monitors anchoring only periodic tree heads.

Limitations (Prototype): - Path-only reconstruction using prefix blocks is not yet active (meta-tree algorithm deferred for correctness); integrity currently relies on full start & end tree recomputation plus prefix subtree matching. - Merkle tree uses copy-up promotion for odd nodes; path algorithm adapted accordingly. ### Proof of Authorization Semantic Validation Modes

The service supports selectable semantic validation tiers for delegation issuance controlled by `AGENTAUTH_POA_VALIDATOR`:

Mode	Env Value	Characteristics
Basic (default)	`basic` or unset	Backwards-compatible minimal invariants: prevent non-wildcard self-delegation, enforce `valid_from < valid_until`, require `jurisdiction` for `regulatory:` scopes, placeholder joint delegation rule (`joint:` requires `signatures` 2), numeric format sanity (`max_amount`, `max_daily_amount`, `currency` if present). Wildcard allowed for self-delegation only.
Advanced	`advanced`	Superset governance rules: mandatory `currency` restriction and 30-day duration cap for any `transaction:` scope, wildcard scope disabled unless `AGENTAUTH_ALLOW_WILDCARD=1`, syntax validation for `valid_hours` (`HH-HH`, wrap-around allowed) and `valid_weekdays` (comma-separated 0–6 unique), optional inline multi-signature restriction unless `AGENTAUTH_ALLOW_INLINE_MULTISIG=1`, aggregate scope length limiting via `AGENTAUTH_MAX_SCOPE_AGG_LEN`, runtime enforcement of `valid_hours` + `valid_weekdays` during token verification.
None	`none`	Disables semantic validation (not recommended for production). Only basic structural field checks apply.

Additional related environment variables:

Env	Purpose
`AGENTAUTH_ALLOW_WILDCARD`	Set to `1` to permit `*` scope under Advanced mode.

Env	Purpose
AGENTAUTH_ALLOW_INLINE_MULTISIG	Set to 1 to allow providing MultiSignatures inline at issuance when threshold > 1.
AGENTAUTH_MAX_SCOPE_AGG_LEN	Integer limit for aggregate length of all scope entries (Advanced).
AGENTAUTH_POA_VALIDATOR	Selects validator mode (advanced, basic, none).

Restriction Keys (current set): currency, jurisdiction, signatures, max_amount, max_daily_amount, valid_hours, valid_weekdays. Future enhancements will add a warning channel and persistent counters for daily limit usage.

Runtime Conditional Enforcement: valid_hours and valid_weekdays are syntactically validated at issuance in Advanced mode and enforced during VerifyToken. Tokens presented outside permitted windows return an unauthorized RFC error (outside permitted hours or outside permitted weekdays). Wrap-around hour windows (e.g., 22-06) are supported.

1.8.3 Full PoA Embedding (RawPOA / PoAVersion)

EnvelopeV2 supports optional embedding of the full canonical PoA JSON for deterministic replay, external audit, and interoperability.

Environment Variables: | Env | Purpose | Default | |—|———|———| | AGENTAUTH_POA_ENVELOPE_V2 | Enables EnvelopeV2 issuance (adds canonical digest & multi-sig metadata). | unset (V1) | | AGENTAUTH_EMBED_FULL_POA | When 1, embeds canonical PoA JSON into RawPOA and sets PoAVersion (poa/v1). | unset (disabled) | | AGENTAUTH_MAX_RAW_POA_BYTES | Max allowed canonical JSON length for embedding; skip & count if exceeded. | 8192 |

Behavior: 1. Canonical JSON produced once via CanonicalPOADigest and reused for RawPOA (ordering preserved: sorted scope & restriction keys; RFC3339 UTC timestamps). 2. Disabled flags or oversize payload RawPOA omitted (token still valid) maintaining backward compatibility. 3. Size violations increment omission counter (envelope_raw_poa_too_large_total). 4. PoAVersion fixed to poa/v1 (future semantic evolution may introduce poa/v2).

Metrics: | Metric | Type | Description | |———|——|————-| | envelope_raw_poa_embedded_total | Counter | Successful embeddings within size cap. | | envelope_raw_poa_too_large_total | Counter | Embedding skipped due to size limit exceeded. |

Operational Guidance: * Track adoption ratio with custom recording rule: embedded_total / envelope_v2_issued_total. * Alert if omission counter spikes (may indicate oversized PoAs or cap tuning need).

Security Notes: * Always verify canonical_digest against recomputed canonical JSON before trusting RawPOA content. * Embedding increases token size; keep cap conservative and benchmark before raising.

Future Roadmap: * CBOR compact encoding (AGENTAUTH_EMBED_FULL_POA_FORMAT=cbor). * Verification helper exposing RawPOA directly (avoid manual envelope parsing). * Streaming / chunking strategy for very large PoAs. * Warning channel & audit persistence of embedded PoA snapshots.

1.8.4 Signed Tree Head (Multi-Sig) & Key Rotation Endpoints

Multi-signature protection provides stronger assurance over append-only revocation transparency by requiring a configured cumulative weight threshold across distinct Ed25519 signing keys.

1.8.4.1 Core Endpoints

Endpoint	Method	Purpose
`/api/v1/crypto/eddsa/keys`	GET	List active & historical Ed25519 public keys (base64url raw 32-byte) with `kid`, optional expiry metadata.
`/api/v1/crypto/rotate`	POST	Force a key rotation (development / testing). Automatically signs latest Merkle tree head post-rotation.
`/api/v1/crypto/ensure-threshold`	POST	Iteratively rotates until satisfied weight configured threshold (bounded attempts). Useful in demos to auto-populate multi-sig state.
`/api/v1/token/revocation/head`	GET	Latest signed tree head (STH) with multi-sig signature array, threshold data and aggregate hash.

1.8.4.2 Signed Tree Head JSON Structure

Example (abridged):

```
{
    "version": 2,
    "merkle_root": "b6f1c9e3dc...",
    "chain_length": 42,
    "aggregate_hash": "e1f2...",              // hash chaining revocation events
    "threshold": 5,                            // required cumulative weight
    "weights_total": 7,                        // sum of all active key weights
    "satisfied_weight": 5,                     // sum of weights of included signatures
    "signatures": [
        {"kid":"ed25519:ab12","alg":"EdDSA","weight":3,"sig":"base64urlSignature1"},
        {"kid":"ed25519:cd34","alg":"EdDSA","weight":2,"sig":"base64urlSignature2"}
    ]
}
```

Fields: | Field | Description | |——-|————-| | `version` | Tree head schema version. | | `merkle_root` | Current Merkle root over all revocation event hashes. | | `chain_length` | Number of revocation events appended. | | `aggregate_hash` | Linear linkage accumulator hash

(integrity chain). | | `threshold` | Required minimum cumulative weight for multi-sig validity. | | `weights_total` | Sum of all active signer weights. | | `satisfied_weight` | Weight achieved by present signatures. | | `signatures[].kid` | Key identifier of signer. | | `signatures[].weight` | Assigned integer weight for signer. | | `signatures[].sig` | Base64url Ed25519 detached signature over canonical STH payload. | | `signatures[].alg` | Algorithm (currently `EdDSA`). |

1.8.4.3 Canonical Signing Payload

`AGENTAUTH_TREE_HEAD:{"version":<int>,"merkle_root":"<hex>","chain_length":<int>,"aggregate_hash`

(Order fixed; no whitespace; numeric values unquoted.)

1.8.4.4 Client Verification Flow (Multi-Sig Panel)

1. Fetch latest STH (`/api/v1/token/revocation/head`).
2. Fetch public keys (`/api/v1/crypto/eddsa/keys`).
3. Reconstruct canonical payload (above) and obtain bytes via UTF-8.
4. Attempt WebCrypto Ed25519 verification (`subtle.verify`).
5. If unsupported, fallback to tweetnacl-based verification via `verifyEd25519` (constant-time-ish detached signature verify). The fallback is injected at runtime and can be overridden via `setEd25519Impl` for testing.
6. Aggregate per-signature results and compute cumulative satisfied weight; display threshold status badge.

Verification Badges: | Badge | Meaning | |——-|———| | `Threshold Met` (green) | `satisfied_weight >= threshold` | | `Partial` (amber) | Not enough cumulative weight yet | | `Verified X/Y` (green) | All Y signatures verified (X = Y) | | `Verified X/Y` (red) | At least one signature failed verification |

Per-Signature Status Cell: | Value | Meaning | |——-|———| | `ok` | Signature verified (WebCrypto or fallback) | | `fail` | Verification failed (error tooltip with reason) | | `n/a` | Missing verification attempt (no signature present) |

Error Codes (tooltips): | Code | Cause | |——|——-| | `missing_public_key` | Key map lacked entry for signer `kid` | | `webcrypto_unsupported` | Browser lacks Ed25519 WebCrypto implementation | | `verify_failed` | WebCrypto verification returned false | | `fallback_unverified` | Fallback verify returned false | | `exception` | Exception thrown during processing |

1.8.4.5 Rotation Semantics

Each successful rotation (development endpoint) triggers automatic signing of the new tree head. This accelerates demo environments by quickly accumulating multi-sig signatures after multiple rotations. Production rotation mechanism would persist prior keys and follow stricter authorization & dual-signature protocols.

1.8.4.6 Environment Variables (Multi-Sig)

Env	Purpose
`AGENTAUTH_MSIG_THRESHOLD`	Numeric threshold required for multi-sig validity.

Env	Purpose
`AGENTAUTH_MSIG_WEIGHTS`	Comma-separated weights for keys in rotation order (e.g. `3,2,1`).
`AGENTAUTH_MSIG_AUTO_SIGN=1`	Auto-sign tree head after any rotation.
`AGENTAUTH_EDDSA_ROTATION_INTERVAL_SEC`	Background rotation interval (if scheduler enabled).
`AGENTAUTH_EDDSA_MAX_HISTORY`	Max historical keys retained/exposed via keys endpoint.

1.8.4.7 Security & UX Notes

- Fallback verification is only used when native Ed25519 unavailable; it relies on audited tweetnacl implementation (public domain). Avoid downgrading silently—panel exposes `webcrypto_unsupported` reason.
- Threshold satisfaction shown independently from signature cryptographic verification; a satisfied threshold with failed signature(s) should be treated as untrusted.
- Canonical payload versioning ensures forward-compatible extensions (additional fields must be added only after version increment).
- For accessibility, badges include `aria-label` and deterministic color contrast; future enhancement will add live region updates.

Roadmap (Multi-Sig): * Dual-signature rotation descriptors anchoring both retiring and successor keys. * External auditor endpoint returning historical STH sequence + batch verification summary. * Path-only consistency proof derivation eliminating full tree rebuild in verification. * Optional signature aggregation (MLS-style) to compress multiple Ed25519 signatures.

1.8.5 Discovery STH Excerpt

`GET /.well-known/agentauth-configuration`


```
{
    "revocation_support": {
    "enabled": true,
    "sth_latest": {
    "version": 2,
    "merkle_root": "b6f1c9e3dc...",
    "chain_length": 42,
    "aggregate_hash": "e1f2...",
    "threshold": 5,
    "weights_total": 7,
    "satisfied_weight": 5,
    "signatures": [
    {"kid": "signerA", "alg": "EdDSA"},
    {"kid": "signerC", "alg": "EdDSA"}
    ]
    }
    }
}
```


Clients must fetch JWKS (`/.well-known/jwks.json`) to obtain Ed25519 public keys for each `kid` and verify multi-signature threshold satisfaction.

1.8.6 JWKS Endpoint

`GET /.well-known/jwks.json`

Returns a JSON Web Key Set containing: - RSA key (RS256) when `AGENTAUTH_USE_JWT_LIB=1` and `AGENTAUTH_JWT_ALG=RS256` - Symmetric placeholder (oct) when only HMAC mode active - Ed25519 public keys as OKP JWK objects when `AGENTAUTH_TOKEN_SIG_MODE=eddsa`

Example response (EdDSA + RS256 hybrid):

```
{
    "keys": [
        {"kty":"RSA","alg":"RS256","kid":"demo-rsa","use":"sig","n":"...","e":"AQAB"},
        {"kty":"OKP","crv":"Ed25519","alg":"EdDSA","kid":"AbCdEf12","use":"sig","x":"base64url
    ]
}
```

Headers: - `ETag`: Weak hash of canonical JWKS JSON for client caching (supports `If-None-Match` → `304`). - `X-JWKS-Signature` + `X-JWKS-Signature-Alg`: Present only when `AGENTAUTH_JWKS_SIGNING_KEY_ENABLED=1` (HMAC-SHA256 integrity hint). - `X-Key-Rotation-Interval-Days`: Optional rotation interval metadata if provided via env.

Rotation Metadata surfaced in discovery (`/.well-known/agentauth-configuration`): - `jwks_etag`: Mirrors current JWKS ETag once fetched at least once. - `jwks_last_rotated`: Timestamp updated whenever ETag changes (new key material).

1.8.7 Notarization Receipt Chain Verification (Prototype)

`GET /api/v1/beta/notarization/receipts/verify`

Verifies integrity of the persisted capability anchor notarization receipt chain.

Response (configured, non-empty, integrity ok):

```
{
    "success": true,
    "configured": true,
    "integrity": "ok",
    "total": 3,
    "chain_head": "b2f4..."
}
```

Response (mismatch detected):

```
{
    "success": true,
    "configured": true,
    "integrity": "mismatch",
    "total": 3,
    "details": {
        "mismatch_index": 0,
        "expected": "0a9c...",      // recomputed chain hash for entry
        "stored": "deadbeef...",     // stored chain_hash field
```

```
        "prev_expected": "",          // previous linkage hash expected
        "prev_stored": ""             // stored prev_hash for entry
    }
}
```

Response (empty chain):

```
{
    "success": true,
    "configured": true,
    "integrity": "empty",
    "total": 0
}
```

Fields: | Field | Description | |——-|————-| | `integrity` | `ok` | `mismatch` | `empty` | | `chain_head` | Last entry `chain_hash` (present only when integrity ok) | | `mismatch_index` | Zero-based index of first invalid entry (only on mismatch) | | `expected` / `stored` | Recomputed vs stored chain hash for failing entry | | `prev_expected` / `prev_stored` | Linkage validation for the failing entry's predecessor |

Related Endpoints: * `GET /api/v1/beta/notarization/receipts/latest` – latest stored receipt object. * `GET /api/v1/beta/notarization/receipts` – list of chain entries with linkage fields.

Metrics Integration: * Gauge: `capability_anchor_notarization_receipts_integrity` (ok=1 mismatch=0 unconfigured/legacy/empty=-1) * Histogram: `capability_anchor_notarization_latency_second` (+ provider labeled variant) * Counter: `capability_anchor_notarization_failures_total` (+ provider labeled variant) * Custom Prometheus Endpoint: `/api/v1/beta/capabilities/anchor/metrics/prome` mirrors the integrity gauge (HELP/TYPE + value) for light scrapes. Force a fresh recomputation with `?verify=1`.

Environment Variables: | Env | Purpose | |—-|———| | `AGENTAUTH_CAP_ANCHOR_NOTARIZE=1` | Enables notarization+receipt path | | `AGENTAUTH_NOTARY_RECEIPT_PERSIST_PATH` | Receipt chain persistence file | | `AGENTAUTH_NOTARY_RECEIPT_VERIFY_INTERVAL` | Background verify loop interval seconds (>=30) | | `AGENTAUTH_CAP_ANCHOR_NOTARY_PROVIDER` | Provider selector (`memory`,`external_stub`) | | `AGENTAUTH_NOTARY_RECEIPT_VERIFY_FRESHNESS_SECONDS` | Max age (seconds) before auto re-verification in custom Prometheus endpoint (default 120) |

Additional Environment Variables (Recent Additions): | Env | Purpose | |—-|———| | `AGENTAUTH_NOTARY_VERIFY_MIN_INTERVAL_SECONDS` | Adaptive verification lower bound (seconds) controlling minimum recomputation cadence | | `AGENTAUTH_NOTARY_VERIFY_MAX_INTERVAL_SECONDS` | Adaptive verification upper bound; interval expands toward this under low append/mismatch rates | | `AGENTAUTH_NOTARY_MERKLE_ENABLED=1` | Enable Merkle root computation per receipt (field `merkle_root` appears) | | `AGENTAUTH_TSA_STUB_MIN_LATENCY_MS` / `AGENTAUTH_TSA_STUB_MAX_LATENCY_MS` | Configure stub TSA latency simulation window | | `AGENTAUTH_TSA_STUB_PROVIDER_NAME` | Override provider name for stub TSA receipts | | `AGENTAUTH_TSA_STUB_POLICY_OID` | Policy OID placeholder for RFC3161 scaffold | | `AGENTAUTH_TLOG_STUB_MIN_LATENCY_MS` / `AGENTAUTH_TLOG_STUB_MAX_LATENCY_MS` | Configure stub transparency log latency simulation window | | `AGENTAUTH_TLOG_STUB_PROVIDER_NAME` | Override provider name for transparency log stub |

1.8.8 External Capability Anchoring (Prototype)

External anchoring publishes the canonical capability registry hash to an external timestamping / transparency provider distinct from the internal memory anchor client and notarization path.

Environment Variables: | Env | Purpose | |—|———| | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_PROVIDER` | Select external provider (`memory` in-process demo or `tsa_stub` latency/failure simulation). | | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_MIN_MS` | Minimum simulated latency (ms) for `tsa_stub` provider (default 25). | | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_MAX_MS` | Maximum simulated latency (ms) for `tsa_stub` provider (default 120). | | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_FAIL_PROB` | Failure probability (0.0–1.0) for `tsa_stub` to exercise retry/error paths (default 0). | | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_RETRIES` | Number of retry attempts after the initial attempt (exponential backoff). Default 0 (no retries). | | `AGENTAUTH_CAP_EXTERNAL_ANCHOR_RETRY_BASE_MS` | Base backoff (ms) before first retry (default 50). Each retry doubles this base (2^n) and applies ±20% jitter. |

Receipt Object (provider-specific, memory & stub share shape):

```
{
    "hash": "sha256:...",
    "timestamp": "2025-10-20T18:59:12.345678901Z",
    "provider": "tsa_stub",
    "version": 1,
    "proof": "..."
}
```

Provider Label Normalization: The metrics label for the TSA stub provider uses a dash (`tsa-stub`) while the environment selector uses an underscore (`AGENTAUTH_CAP_EXTERNAL_ANCHOR_PROVIDER=tsa_stub`). Internally the provider implementation emits `provider:"tsa-stub"` in receipts; the metrics helper normalizes the env value `tsa_stub` → `tsa-stub` to ensure consistent labeled counters/histograms. Memory provider passes through unchanged (`memory`). Custom providers should prefer dash-separated labels for Prometheus cardinality hygiene.

Endpoints: | Endpoint | Method | Purpose | |———-|———|———| | `/api/v1/beta/capabilities/anchor/statu` | GET | Includes `external_anchor_receipt` when at least one external anchor succeeded. | | `/api/v1/beta/capabilities/anchor/verify` | POST | Verifies a posted receipt object (hash/timestamp/provider) against provider logic. | | `/api/v1/beta/capabilities/anchor/external/receipts/` | GET | Latest persisted external anchor receipt (with `prev_hash` and `chain_hash`). | | `/api/v1/beta/capabilities/anchor/external/receipts` | GET | Full external anchor receipt chain (append-only order). | | `/api/v1/beta/capabilities/anchor/external/receipts/verify` | GET | External receipt chain integrity verification (reports ok | mismatch | empty plus diagnostics). |

Metrics Adapter Injection: To ensure the initial external anchoring attempt (performed during server startup when a capability registry hash is already available) records into a custom metrics backend, use the new constructor:

```
reg := prometheus.NewRegistry()
pm  := metrics.NewPrometheusMetrics(metrics.PrometheusAdapterOptions{Namespace: "agentauth", Su
srv := web.NewBetaServerWithMetrics(":0", pm)
```

This avoids the earlier pattern of constructing with `NewBetaServer` then replacing `srv.metrics`

(which missed startup side-effects such as the initial external anchor attempt). Tests targeting strict Prometheus assertions should prefer `NewBetaServerWithMetrics`. | `/api/v1/beta/capabilities/anchor/exter` | GET | Latest external anchor receipt only (configured provider). | | `/api/v1/beta/capabilities/anchor/exter` | GET | Performs provider-level verification of latest receipt (`verified:true|false`). |

Metrics (Prometheus Adapter): | Metric | Type | Labels | Description | |———|———|———|————-| | `external_anchor_attempts_total` | Counter | `provider` | Total external anchoring attempts initiated. | | `external_anchor_failures_total` | Counter | `provider` | Failed anchoring attempts. | | `external_anchor_latency_seconds` | Histogram | `provider` | Latency distribution of successful external anchoring operations. | | `external_anchor_age_seconds` | Gauge | none | Seconds since last successful external anchor (0 when never). | | `external_anchor_last_hash_len` | Gauge | none | Length of last externally anchored hash (0 when none). | | `capability_external_anchor_receipts_integrity` | Gauge | none | External receipt chain integrity (1=ok,0=mismatch,-1=unconfigured/empty). | | `capability_external_anchor_receipts_last_verify_age_seconds` | Gauge | none | Seconds since last external receipt chain integrity verification. | | `capability_external_anchor_receipts_total` | Counter | none | Total persisted external anchor receipt entries. |

Startup Behavior: An initial anchoring attempt is performed immediately during server initialization (if a capability registry hash already exists) to ensure observability for static demo seeds.

Failure Simulation: Retry Behavior: If `AGENTAUTH_CAP_EXTERNAL_ANCHOR_RETRIES` > 0, the server performs additional attempts upon failure using exponential backoff: `delay = base_ms * 2^attempt` with a $\pm 20\%$ jitter adjustment. Each attempt (initial + retries) records its own metrics (attempt counter increment; failures or latency histogram sample). Successful attempts short-circuit remaining retries. The hash length gauge is updated only on success. Provider-labeled counters reflect total attempts and failures across all retries. Set `AGENTAUTH_CAP_EXTERNAL_ANCHOR_PROVIDER=tsa_stub` with `AGENTAUTH_CAP_EXTERNAL_ANCHOR_FAIL_PROB=1` to force failures; status payload will omit `external_anchor_receipt` and failure counter increments.

Verification Semantics: `/external/verify` invokes provider `Verify`; memory provider always succeeds when hash matches, while stub may return errors if simulated state diverges.

Observability Guidance: 1. Scrape `/metrics` for labeled counters/histogram buckets. 2. Monitor `external_anchor_age_seconds` for freshness. 3. Alert on high failure ratio; consider provider fallback.

Roadmap: * Add real TSA / transparency log provider implementation (RFC3161, RFC6962). * (Completed) Persist external receipts with hash-chain linkage similar to notarization chain (see new `/external/receipts` endpoints & integrity metrics). * Integrate multi-provider quorum anchoring. * Export proof verification / inclusion path endpoints when transparency log present.

Receipt Object Extensions: Unified Recording Helper: When instrumenting a single external anchoring operation manually (e.g. custom provider integration code outside the server), prefer the consolidated helper to ensure unlabeled and provider-labeled metrics remain synchronized:

```
// success path
pm.RecordExternalAnchorResult(providerTag, true, latency, len(hashHex))
// failure path
pm.RecordExternalAnchorResult(providerTag, false, 0, 0)
```

Behavior: * Always increments attempts counters (unlabeled + provider). * On success: observes latency (both histograms) and sets last hash length gauge. * On failure: increments failure counters (unlabeled + provider). * Does NOT set age gauge directly (background server loop handles freshness).

Backward compatibility: existing `IncExternalAnchorAttempts`, `IncExternalAnchorFailures`, `ObserveExternalAnchorLatency`, and `SetExternalAnchorLastHashLen` remain available; the server now prefers the unified helper when present.

The persisted receipt entries (via `/api/v1/beta/notarization/receipts` and latest endpoint) may now include optional fields: | Field | Description | |——-|————-| | `merkle_root` | Present when Merkle feature enabled; binary Merkle tree root over all leaf entry digests up to this entry. Leaves = sha256 of base JSON (excluding `chain_hash`, `merkle_root`). | | `rotation` | Key rotation descriptor if the receipt represents a rotation event (continuity metadata). |

Key Rotation Descriptor (`rotation`):

```
"rotation": {
    "old_key_id": "ed25519:abcd...",
    "new_key_id": "ed25519:ef01...",
    "effective_time": "2025-10-20T12:00:00Z",
    "reason": "scheduled",
    "prev_rotation_hash": "<chain_hash_of_previous_rotation_receipt>"
}
```

Purpose: preserve continuity and audit trail of signing keys; future enhancement will require dual signatures (old + new key) over descriptor. ### Key Rotation Verification (Prototype)

`GET /api/v1/beta/rotations/verification`

Audits continuity (`prev_rotation_hash` linkage) and dual-signature validity of recorded key rotation descriptors embedded in notarization receipts.

Example Response (abridged):

```
{
    "success": true,
    "configured": true,
    "generated_at": "2025-10-20T12:34:56Z",
    "summary": {
        "total": 2,
        "all_continuity_ok": true,
        "all_signatures_ok": false,
        "failures": 1,
        "results": [
            {"index":0,"old_key_id":"ed25519:aa12bb34","new_key_id":"ed25519:cc56dd78","continu
            {"index":1,"old_key_id":"ed25519:cc56dd78","new_key_id":"ed25519:ee90ff11","continu
        ]
    }
}
```

Per-Descriptor Fields: | Field | Description | |——-|————-| | `index` | Sequence position (0-based) | | `old_key_id` / `new_key_id` | Key identifiers bound to descriptor | | `continuity_ok` |

Whether `prev_rotation_hash` matches previous rotation receipt hash (empty allowed for first) | | `signatures_ok` | Both old & new key signatures present and verified | | `reason` | First failure reason (present only on failure) | | `prev_hash_expected` / `prev_hash_actual` | Continuity diagnostics when prior hash known |

Aggregate Fields: | Field | Description | |——-|————-| | `total` | Number of descriptors evaluated | | `all_continuity_ok` | No continuity failures encountered | | `all_signatures_ok` | All dual signatures validated | | `failures` | Count of descriptors with any failure |

Failure Reasons: | Reason | Meaning | |———|———| | `descriptor_nil` | Nil descriptor entry | | `continuity_failure` | Previous linkage mismatch or unexpected non-empty genesis prev hash | | `kid_not_found_old` | Old key public key material unavailable | | `kid_not_found_new` | New key public key material unavailable | | `kid_mismatch_old` | Old key ID mismatch vs derived ID | | `kid_mismatch_new` | New key ID mismatch vs derived ID | | `missing_old_signature` | Old key signature field empty | | `missing_new_signature` | New key signature field empty | | `old_sig_invalid` | Old signature fails Ed25519 verify / decode | | `new_sig_invalid` | New signature verification failure | | `serialization_error` | Canonical payload marshal failed |

Metrics: | Metric | Description | |———|————-| | `agentauth_rotation_verification_latency_seconds` | Histogram of verification latency | | `agentauth_rotation_verification_total{outcome="success|failure"}` | Counter labeled by outcome (any failure => failure) | | `agentauth_rotation_verification_failure_reason_to` | Counter per failing descriptor categorized by first failure reason |

Current Limitations: * Public key resolution not yet wired—missing keys produce `kid_not_found_old/new`. * Rotation descriptors must be embedded in receipts; external persistence forthcoming. * No signed summary artifact yet (planned for hardening phase).

Roadmap: * Persistent rotation receipt log with independent integrity chain. * RFC3161/Sigstore timestamping of rotation chain heads. * Inclusion proofs & Merkle subtree optimization for descriptor sets.

Stub Provider Interfaces: - TSA Scaffold (`internal/notary/tsa_stub.go`): `Timestamp` method returns synthetic `TSAResponse` with `emulated_serial` and `emulated_policy_oid`. - Transparency Log Scaffold (`internal/notary/transparency_log_stub.go`): `Log` method returns synthetic `TransparencyLogEntryResponse` with `emulated_leaf_id`, `emulated_root_ref`.

Adaptive Verification Notes: - Background verification interval dynamically recalculates between min/max bounds based on append rate and mismatch occurrences. - Force immediate recomputation via custom Prometheus endpoint `?verify=1` or when freshness threshold exceeded. - Gauge `capability_anchor_notarization_receipts_last_verify_age_seconds` exposes age since last integrity verification; alert if near max interval * 2.

Merkle Root Verification (Auditor Guidance): 1. Fetch `/api/v1/beta/notarization/receipts`. 2. For each entry reconstruct leaf digest (sha256 over base JSON minus chain fields) maintaining ordered list. 3. Recompute Merkle root using pairwise hashing duplicating last node when level has odd count; compare to final entry's `merkle_root`. 4. Cross-check final `chain_hash` for chain linkage integrity.

Rotation Audit Flow (Future): 1. Identify all receipts with `rotation` present. 2. Verify linear continuity via `prev_rotation_hash` forming rotation chain. 3. Validate dual signatures over descriptor referencing both key IDs.

Dual-Signature Fields (Implemented): | Field | Description | |——-|————-| | `old_key_signature`

| Ed25519 signature produced by retiring key over canonical, prefixed descriptor payload | | `new_key_signature` | Ed25519 signature produced by successor key over same canonical payload |

Verification Failure Codes: | Code | Meaning | |——|———| | `kid_mismatch_old` | Provided old public key does not derive same key id | | `kid_mismatch_new` | Provided new public key does not derive same key id | | `missing_old_signature` | Old key signature absent | | `missing_new_signature` | New key signature absent | | `old_sig_invalid` | Old signature fails verification | | `new_sig_invalid` | New signature fails verification | | `serialization_error` | Canonical descriptor could not be serialized |

Canonical Signing Payload:

`AGENTAUTH_ROTATION_DESCRIPTOR:{"old_key_id":"...","new_key_id":"...","effective_time":"...","re`

1.8.9 Snapshot Metrics & Integrity

Metrics exposed during snapshot operations (Prometheus): | Metric | Type | Labels | Description | |———|——|———|————-| | `agentauth_snapshot_generation_latency_seconds` | Histogram | - | Generation latency | | `agentauth_snapshot_generation_total` | Counter | outcome | Attempts (success|error) | | `agentauth_snapshot_verification_latency_seconds` | Histogram | - | Verification latency | | `agentauth_snapshot_verification_total` | Counter | outcome | Attempts (success|failure|error) |

Snapshot Verification Result Fields: | Field | Meaning | |——-|———| | `valid` | All comparisons succeeded (hash, merkle, chain head, count) | | `hash_match` | Snapshot self-hash matches recomputed digest | | `merkle_match` | Merkle root matches recomputed tree (or absent when disabled) | | `chain_head_match` | Chain head hash equals current store head | | `receipt_count_ok` | Receipt count equals current store length | | `reason` | First failing condition (precedence: merkle>hash>chain_head>receipt_count) |

Exit Codes (CLI): reference README section for mapping (0 success; non-zero failure/error categories).

Alerting Recommendation: Trigger alert when integrity gauge == 0 (mismatch) for 2 consecutive scrapes. Suppress alerts for value -1 (unconfigured/empty/legacy).

1.8.10 JTI TTL & Replay Protection

The token service enforces fail-closed replay detection on the `jti` claim. In-memory JTIs expire after a TTL window: - Configure with `AGENTAUTH_JTI_TTL_SEC` (default 600 seconds). - On validation, a duplicate JTI within the TTL window token rejected. - Opportunistic eviction runs when the in-memory map exceeds 500 entries. - Pluggable persistence layer can be injected via `WithReplayStore` to enforce durable uniqueness.

Edge Cases: - Expired JTI entries (older than TTL) are removed lazily, allowing reuse (treated as a fresh token). - Absent or empty `jti` immediate rejection.

Security Considerations: - For production deployments use a durable store (e.g., Redis) implementing `ReplayStore` to survive restarts and coordinate across replicas. - TTL tuning balances memory footprint vs. replay window risk; choose a value aligned with average token lifetime. — Need context? See: README.md | docs/ARCHITECTURE.md | docs/GETTING_STARTED.md # (Metadata moved to top) AgentAuth RFC API Reference

Last Updated: 2025-10-21 Status: Active

Beta reference – NOT production ready. Interfaces and examples may be incomplete or simplified. See `../DISCLAIMER.md` for intentionally missing security, policy, and lifecycle controls.

Beta Demonstration API Documentation (NOT Production Ready)

Complete Go library API reference for AAP-001 (AgentAuth 1.0) and AAP-002 (PoA Definition) implementation.

Note: This document covers the Go library API. For the web demonstration API documentation, see COMPLETE_API_REFERENCE.md which includes both library and web API documentation.

1.9 Table of Contents

Web API: For REST API endpoints used by the webapp demo, see COMPLETE_API_REFERENCE.md

1.10 Core Service API

1.10.1 RFCCompliantService

The main service implementing both AAP-001 and AAP-002 specifications.

```
type RFCCompliantService struct {
    jwtService         *ProperJWTService
    legalValidator     *LegalFrameworkValidator
    delegationManager  *DelegationManager
    attestationService *AttestationService
}
```

1.10.1.1 Constructor

```
func NewRFCCompliantService(issuer, audience string) (*RFCCompliantService, error)
```

Parameters: - `issuer` (string): The issuer identifier for JWT tokens - `audience` (string): The intended audience for JWT tokens

Returns: - `*RFCCompliantService`: Configured service instance - `error`: Configuration or initialization error

Example:

```
service, err := auth.NewRFCCompliantService("my-company", "ai-authorization")
### Signature & Semantic Validation Options (AAP-001 Service)

The lower-level `aap001.Service` (delegation issuance / token verification) now supports additi

```go
// Enforce signature presence (abort issuance if signing fails)
func WithMandatorySignatures() aap001.Option
// Provide a signing key (returned Signer must implement ed25519 or supported algorithm)
func WithSignerProvider(fn func() (cr.Signer, error) aap001.Option
// Install semantic validator executing cross-field invariants before signing/persistence
func WithSemanticValidator(v aap001.PoAValidator) aap001.Option
// Enforce strict authenticity (missing public key at verification becomes integrity failure)
func WithStrictAuthenticity() aap001.Option
```

Canonical Digest & Signature:

```
digestHex, canonicalJSON, err := aap001.CanonicalPOADigest(&poa)
// Signature covers domain-separated canonical JSON; exclusion of mutable fields preserves val
```

Validator Interface:

```
type PoAValidator interface {
    Validate(*aap001.PowerOfAttorney) error // return rfc.ErrInvalidRequest on semantic violat
}

// Provided prototype implementation:
type BasicPoAValidator struct{}
```

Prototype Semantic Rules (BasicPoAValidator): 1. Disallow self-delegation unless wildcard-only scope. 2. Require `currency` restriction for any `transaction:` scope. 3. Enforce temporal ordering (`valid_from` < `valid_until`). 4. Cap `transaction:` delegation duration to 30 days. 5. Normalize nil restriction map.

Activation Example:

```
svc := aap001.NewService(audit.NewMemoryLogger(nil), authz.NewMemoryAuthorizer(),
    aap001.WithSignerProvider(func() (cr.Signer, error) { kp, _ := cr.NewInMemoryEd25519Provide
    aap001.WithMandatorySignatures(),
    aap001.WithSemanticValidator(aap001.BasicPoAValidator{}),
)
```

Token Verification now sets `SignatureValid` and `PublicKeyFound` flags in `TokenVerificationResult` if signature present.
```

RawPOA Exposure: `TokenVerificationResult` now includes `raw_poa` and `poa_version` fields when a token was issued as EnvelopeV2 with full PoA embedding enabled and within size limits. These fields are omitted when embedding disabled or size cap exceeded. Always verify `canonical_digest` independently before trusting embedded JSON. Future versions may add `raw_poa_cbor` for compact representation.

if err != nil { return fmt.Errorf("service creation failed: %w", err) }

```
##  **AAP-001 Authorization API**

### **AuthorizeAgentAuth**

Main authorization method implementing complete AAP-001 flow with AAP-002 PoA Definition valida

```go
func (s *RFCCompliantService) AuthorizeAgentAuth(ctx context.Context, req AgentAuthRequest) (*
```

Parameters: - `ctx` (context.Context): Request context for cancellation and timeout - `req` (AgentAuthRequest): Complete RFC-compliant authorization request

Returns: - `*AgentAuthResponse`: Authorization response with compliance validation - `error`: Authorization or validation error

Process Flow: 1. Validates PoA Definition (AAP-002) 2. Validates principal capacity (AAP-001) 3. Validates AI client capabilities 4. Validates legal compliance 5. Generates authorization code 6. Creates comprehensive audit record

Example:

```go
response, err := service.AuthorizeAgentAuth(ctx, auth.AgentAuthRequest{
    ClientID:     "ai_agent_v1",
    ResponseType: "code",
    Scope:        []string{"financial_advisory"},
    PowerType:    "financial_advisory_powers",
    PrincipalID:  "corp_ceo_123",
    AIAgentID:    "ai_financial_advisor",
    Jurisdiction: "US",
    PoADefinition: poaDefinition, // Complete AAP-002 structure
})
```

1.10.2 CreateAgentAuthToken (Future Enhancement)

Exchange authorization code for extended token with comprehensive metadata.

```go
func (s *RFCCompliantService) CreateAgentAuthToken(ctx context.Context, authCode string) (*Age
```

1.11 AAP-002 PoA Definition API

1.11.1 PoA Definition Structure

Complete implementation of AAP-002 Power-of-Attorney Credential Definition.
```

```
type PoADefinition struct {
    Principal         Principal         `json:"principal"`          // Section 3.A
    Authorizer        Authorizer        `json:"authorizer"`         // Section 3.A
    Client            ClientAI          `json:"client"`             // Section 3.A
    AuthorizationType AuthorizationType `json:"authorization_type"` // Section 3.B
    ScopeDefinition   ScopeDefinition   `json:"scope_definition"`   // Section 3.B
    Requirements      Requirements      `json:"requirements"`       // Section 3.C
}
```

1.11.2 Section A: Parties

1.11.2.1 Principal

```
type Principal struct {
    Type         PrincipalType `json:"type"`                    // "individual" or "organizati
    Identity     string        `json:"identity"`                // Unique identifier
    Organization *Organization `json:"organization,omitempty"`  // Required if type is "organi
}

type PrincipalType string
const (
    PrincipalTypeIndividual   PrincipalType = "individual"
    PrincipalTypeOrganization PrincipalType = "organization"
)
```

1.11.2.2 Organization

```
type Organization struct {
    Type                OrganizationType `json:"type"`                 // Commercial, public,
    Name                string           `json:"name"`                 // Organization name
    RegisterEntry       string           `json:"register_entry"`       // Commercial register
    ManagingDirector    string           `json:"managing_director"`    // Current managing dir
    RegisteredAuthority bool             `json:"registered_authority"` // Has registered autho
}

type OrganizationType string
const (
    OrgTypeCommercial   OrganizationType = "commercial_enterprise"    // AG, Ltd., partnership
    OrgTypePublic       OrganizationType = "public_authority"         // Federal, state, munic
    OrgTypeNonProfit    OrganizationType = "non_profit_organization"  // Foundation, gGmbH
    OrgTypeAssociation  OrganizationType = "association"              // Non-profit or non-cha
    OrgTypeOther        OrganizationType = "other"                    // Church, cooperative,
)
```

1.11.2.3 ClientAI

```
type ClientAI struct {
    Type              ClientType `json:"type"`              // LLM, agent, agentic AI, robot
    Identity          string     `json:"identity"`          // Unique AI identifier
```

```
    Version           string     `json:"version"`            // AI version
    OperationalStatus string     `json:"operational_status"` // "active", "revoked", etc.
}

type ClientType string
const (
    ClientTypeLLM       ClientType = "llm"            // Large Language Model
    ClientTypeAgent     ClientType = "digital_agent"  // Single digital agent
    ClientTypeAgenticAI ClientType = "agentic_ai"     // Team of agents
    ClientTypeRobot     ClientType = "humanoid_robot" // Physical humanoid robot
    ClientTypeOther     ClientType = "other"          // Other AI types
)
```

1.11.3 Section B: Authorization Type & Scope

1.11.3.1 AuthorizationType

```
type AuthorizationType struct {
    RepresentationType    RepresentationType `json:"representation_type"`     // Sole or joint
    RestrictionsExclusions []string          `json:"restrictions_exclusions"` // Specific excl
    SubProxyAuthority     bool              `json:"sub_proxy_authority"`     // Can delegate f
    SignatureType         SignatureType     `json:"signature_type"`          // Single, joint,
}

type RepresentationType string
const (
    RepresentationSole  RepresentationType = "sole"  // Sole representation
    RepresentationJoint RepresentationType = "joint" // Joint representation
)

type SignatureType string
const (
    SignatureSingle     SignatureType = "single"     // Single signature authority
    SignatureJoint      SignatureType = "joint"      // Joint signature required
    SignatureCollective SignatureType = "collective" // Collective signature required
)
```

1.11.3.2 ScopeDefinition

```
type ScopeDefinition struct {
    ApplicableSectors  []IndustrySector   `json:"applicable_sectors"`  // ISIC/NACE industry c
    ApplicableRegions  []GeographicScope  `json:"applicable_regions"`  // Geographic constrain
    AuthorizedActions  AuthorizedActions  `json:"authorized_actions"`  // Permitted actions
}
```

1.11.3.3 Industry Sectors (ISIC/NACE)

```
type IndustrySector string
const (
```

```
    SectorAgriculture     IndustrySector = "agriculture_forestry_fishing"
    SectorMining          IndustrySector = "mining_quarrying"
    SectorManufacturing   IndustrySector = "manufacturing"
    SectorEnergy          IndustrySector = "energy_supply"
    SectorWater           IndustrySector = "water_supply"
    SectorWaste           IndustrySector = "waste_management"
    SectorConstruction    IndustrySector = "construction"
    SectorTrade           IndustrySector = "trade"
    SectorTransport       IndustrySector = "transport_storage"
    SectorHospitality     IndustrySector = "hospitality"
    SectorICT             IndustrySector = "information_communication"
    SectorFinancial       IndustrySector = "financial_insurance"
    SectorRealEstate      IndustrySector = "real_estate"
    SectorProfessional    IndustrySector = "professional_scientific"
    SectorBusiness        IndustrySector = "business_services"
    SectorPublicAdmin     IndustrySector = "public_administration"
    SectorEducation       IndustrySector = "education"
    SectorHealth          IndustrySector = "health_social_work"
    SectorArts            IndustrySector = "arts_entertainment"
    SectorOtherServices   IndustrySector = "other_services"
)
```

1.11.3.4 Geographic Scope

```
type GeographicScope struct {
    Type        GeographicType `json:"type"`                   // Global, national, regional,
    Identifier  string         `json:"identifier"`             // Country code, region name, e
    Description string         `json:"description,omitempty"`  // Human-readable description
}

type GeographicType string
const (
    GeoTypeGlobal      GeographicType = "global"           // Global operations
    GeoTypeNational    GeographicType = "national"         // Specific country (specify identi
    GeoTypeRegional    GeographicType = "regional"         // DACH, Benelux, NAFTA, etc.
    GeoTypeSubnational GeographicType = "subnational"      // States, provinces, municipalitie
    GeoTypeSpecific    GeographicType = "specific_location" // Specific branches or locations
)
```

1.11.3.5 Authorized Actions

```
type AuthorizedActions struct {
    Transactions      []TransactionType   `json:"transactions"`        // Financial transactio
    Decisions         []DecisionType      `json:"decisions"`           // Business decisions
    NonPhysicalActions []NonPhysicalAction `json:"non_physical_actions"` // Information action
    PhysicalActions   []PhysicalAction    `json:"physical_actions"`    // Physical world actio
}
```

```go
// Transaction Types
type TransactionType string
const (
    TransactionLoan     TransactionType = "loan_transactions"
    TransactionPurchase TransactionType = "purchase_transactions"
    TransactionSale     TransactionType = "sale_transactions"
    TransactionLease    TransactionType = "leasing_rental"
)

// Decision Types
type DecisionType string
const (
    DecisionPersonnel    DecisionType = "personnel_decisions"    // Hiring, dismissal, develop
    DecisionFinancial    DecisionType = "financial_commitments"  // Contracts, expenses, inves
    DecisionBuySell      DecisionType = "buy_sell_transactions"  // Asset acquisition/disposal
    DecisionConceptual   DecisionType = "conceptual_determinations" // Business models, concep
    DecisionDesign       DecisionType = "design_decisions"       // Branding, architecture
    DecisionInformation  DecisionType = "information_sharing"    // Data disclosure, PR
    DecisionStrategic    DecisionType = "strategic_decisions"    // M&A, partnerships, strateg
    DecisionLegal        DecisionType = "legal_proceedings"      // Legal actions
    DecisionAsset        DecisionType = "asset_management"       // Asset management decisions
)

// Non-Physical Actions
type NonPhysicalAction string
const (
    ActionSharing      NonPhysicalAction = "sharing_presenting"
    ActionBrainstorm   NonPhysicalAction = "brainstorming"
    ActionResearch     NonPhysicalAction = "researching_rag"    // Including RAG operations
)

// Physical Actions (primarily for humanoid robots)
type PhysicalAction string
const (
    ActionShipment     PhysicalAction = "shipments"      // Ocean, air, truck shipments
    ActionProduction   PhysicalAction = "production"     // Manufacturing processes (for demon
    ActionRecycling    PhysicalAction = "recycling"      // Recycling operations
    ActionStorage      PhysicalAction = "storage"        // Physical storage operations
    ActionCustomization PhysicalAction = "customization" // Product customization
    ActionPackage      PhysicalAction = "package"        // Packaging operations
    ActionClean        PhysicalAction = "clean"          // Cleaning operations
)
```

1.11.4 Section C: Requirements

1.11.4.1 Requirements Structure

```
type Requirements struct {
    ValidityPeriod    ValidityPeriod    `json:"validity_period"`     // Time constraints
    FormalRequirements FormalRequirements `json:"formal_requirements"` // Legal formalities
    PowerLimits       PowerLimits       `json:"power_limits"`        // Authority limitations
    SpecificRights    SpecificRights    `json:"specific_rights"`     // Rights and obligations
    SpecialConditions SpecialConditions `json:"special_conditions"`  // Special conditions
    DeathIncapacity   DeathIncapacity   `json:"death_incapacity"`    // Death/incapacity rules
    SecurityCompliance SecurityCompliance `json:"security_compliance"` // Security requirement
    JurisdictionLaw   JurisdictionLaw   `json:"jurisdiction_law"`    // Legal framework
    ConflictResolution ConflictResolution `json:"conflict_resolution"` // Dispute resolution
}
```

1.11.4.2 ValidityPeriod

```
type ValidityPeriod struct {
    StartTime       time.Time      `json:"start_time"`        // Authorization start time
    EndTime         time.Time      `json:"end_time"`          // Authorization end time (max 1
    TimeWindows     []TimeWindow   `json:"time_windows"`      // Operational time windows
    GeoConstraints  []string       `json:"geo_constraints"`   // Geographic restrictions
    SuspensionRules []string       `json:"suspension_rules"`  // Automatic suspension conditio
}

type TimeWindow struct {
    Start    string   `json:"start"`    // Start time (HH:MM format)
    End      string   `json:"end"`      // End time (HH:MM format)
    Timezone string   `json:"timezone"` // Timezone identifier
    Days     []string `json:"days"`     // Days of week (Mon, Tue, etc.)
}
```

1.11.4.3 PowerLimits

```
type PowerLimits struct {
    PowerLevels        []PowerLevel     `json:"power_levels"`         // Amount and transactio
    InteractionBoundaries []string      `json:"interaction_boundaries"` // Data access, collab
    ToolLimitations    []string         `json:"tool_limitations"`     // Permitted tools and A
    OutcomeLimitations []string         `json:"outcome_limitations"`  // Intended outcome cons
    ModelLimits        []ModelLimit     `json:"model_limits"`         // AI model restrictions
    BehavioralLimits   []string         `json:"behavioral_limits"`    // Action restrictions
    QuantumResistance  bool             `json:"quantum_resistance"`   // Require quantum-resis
    ExplicitExclusions []string         `json:"explicit_exclusions"`  // Explicitly forbidden
}

type PowerLevel struct {
    Type        string  `json:"type"`        // "amount", "transaction_type", etc.
    Limit       float64 `json:"limit"`       // Numerical limit value
```

```
    Currency    string  `json:"currency"`    // Currency code (if applicable)
    Description string  `json:"description"` // Human-readable description
}

type ModelLimit struct {
    ParameterCount   int64    `json:"parameter_count"`   // Maximum model parameters
    ReasoningMethods []string `json:"reasoning_methods"` // Permitted reasoning methods
    TrainingMethods  []string `json:"training_methods"`  // Permitted training approaches
    Description      string   `json:"description"`       // Limit description
}
```

1.11.4.4 JurisdictionLaw

```
type JurisdictionLaw struct {
    Language           string   `json:"language"`             // Contract language
    GoverningLaw       string   `json:"governing_law"`        // Applicable legal framework
    PlaceOfJurisdiction string  `json:"place_of_jurisdiction"` // Legal jurisdiction
    AttachedDocuments  []string `json:"attached_documents"`   // Referenced legal documents
}
```

1.12 Professional Foundation API

1.12.1 ProperJWTService

Development JWT implementation with RSA-256 signatures.

```
type ProperJWTService struct {
    privateKey *rsa.PrivateKey
    publicKey  *rsa.PublicKey
    issuer     string
    audience   string
}

func NewProperJWTService(issuer, audience string) (*ProperJWTService, error)
func (s *ProperJWTService) CreateToken(subject string, scope []string, expiry time.Duration) (s
func (s *ProperJWTService) ValidateToken(tokenString string) (*CustomClaims, error)
```

1.12.2 LegalFrameworkValidator

Multi-jurisdiction legal validation.

```
type LegalFrameworkValidator struct {
    supportedJurisdictions map[string]bool
    supportedEntityTypes   map[string]bool
    complianceRules        map[string][]string
}

func (v *LegalFrameworkValidator) ValidateFramework(ctx context.Context, framework LegalFramewo
```

1.13 Response Types

1.13.1 AgentAuthResponse

```
type AgentAuthResponse struct {
    AuthorizationCode string              `json:"code"`              // OAuth authorization co
    State             string              `json:"state"`             // CSRF protection state
    ExtendedToken     string              `json:"extended_token"`    // Optional immediate toke
    LegalCompliance   bool                `json:"legal_compliance"`  // Legal validation result
    AuditRecordID     string              `json:"audit_record_id"`   // Audit trail identifier
    ExpiresIn         int                 `json:"expires_in"`        // Code expiration (second
    Scope             []string            `json:"scope"`             // Granted scopes
    PoAValidation     PoAValidationResult `json:"poa_validation"`    // PoA validation details
}
```

1.13.2 PoAValidationResult

```
type PoAValidationResult struct {
    Valid             bool     `json:"valid"`              // Overall validation result
    ValidationErrors  []string `json:"validation_errors"`  // Specific validation errors
    ComplianceLevel   string   `json:"compliance_level"`   // "AAP-002_compliant", etc.
    AttestationStatus string   `json:"attestation_status"` // "validated", "pending", etc.
}
```

1.13.3 AgentAuthToken

```
type AgentAuthToken struct {
    AccessToken      string           `json:"access_token"`      // JWT access token
    TokenType        string           `json:"token_type"`        // "bearer"
    ExpiresIn        int              `json:"expires_in"`        // Token expiration (seconds)
    Scope            []string         `json:"scope"`             // Token scopes
    ExtendedMetadata ExtendedMetadata `json:"extended_metadata"` // PoA metadata
}

type ExtendedMetadata struct {
    PowerType        string        `json:"power_type"`         // Type of power granted
    PrincipalID      string        `json:"principal_id"`       // Principal identifier
    AIAgentID        string        `json:"ai_agent_id"`        // AI agent identifier
    PoADefinition    PoADefinition `json:"poa_definition"`     // Complete PoA definition
    AttestationLevel string        `json:"attestation_level"`  // Attestation level achieved
    ComplianceProof  []string      `json:"compliance_proof"`   // Compliance validation proof
}
```

1.14 Error Handling

1.14.1 Error Types

```
// RFC validation errors
type RFCValidationError struct {
```

```
    Code    string `json:"code"`    // Error code (e.g., "invalid_principal")
    Message string `json:"message"` // Human-readable message
    Field   string `json:"field"`   // Field that caused the error
}

// Legal compliance errors
type LegalComplianceError struct {
    Jurisdiction string `json:"jurisdiction"` // Jurisdiction where error occurred
    Regulation   string `json:"regulation"`   // Specific regulation violated
    Description  string `json:"description"`  // Error description
}

// AI capability errors
type AICapabilityError struct {
    ClientID   string `json:"client_id"`   // AI client identifier
    Capability string `json:"capability"`  // Missing capability
    PowerType  string `json:"power_type"`  // Power type requiring capability
}
```

1.14.2 Common Error Codes

Error Code	Description	HTTP Status
`invalid_request`	Malformed request structure	400
`invalid_principal`	Principal validation failed	400
`invalid_client`	AI client validation failed	400
`unsupported_jurisdiction`	Jurisdiction not supported	400
`insufficient_capabilities`	AI lacks required capabilities	403
`excessive_power_limits`	Requested powers exceed limits	403
`legal_compliance_failure`	Legal framework validation failed	403
`invalid_poa_definition`	PoA Definition validation failed	400

1.15 Usage Examples

1.15.1 Complete RFC Implementation Example

```
package main

import (
    "context"
    "fmt"
    "time"
    "github.com/agentauth/agentauth/pkg/auth"
)

func main() {
    // Create service
    service, err := auth.NewRFCCompliantService("GlobalTech", "ai-authorization")
```

```
if err != nil {
    panic(err)
}

// Create comprehensive PoA Definition
poa := auth.PoADefinition{
    // A. Parties
    Principal: auth.Principal{
        Type:     auth.PrincipalTypeOrganization,
        Identity: "GlobalTech-Corp-2025",
        Organization: &auth.Organization{
            Type:                auth.OrgTypeCommercial,
            Name:                "GlobalTech Corporation",
            RegisterEntry:       "HRB-123456",
            ManagingDirector:    "Dr. Sarah Johnson",
            RegisteredAuthority: true,
        },
    },
    Client: auth.ClientAI{
        Type:              auth.ClientTypeAgenticAI,
        Identity:          "ai_financial_advisor_v3",
        Version:           "3.2.1-prod", # (for demonstration only; NOT production ready)
        OperationalStatus: "active",
    },

    // B. Authorization Type & Scope
    AuthorizationType: auth.AuthorizationType{
        RepresentationType: auth.RepresentationSole,
        SignatureType:      auth.SignatureSingle,
        SubProxyAuthority:  false,
    },
    ScopeDefinition: auth.ScopeDefinition{
        ApplicableSectors: []auth.IndustrySector{
            auth.SectorFinancial, auth.SectorICT,
        },
        ApplicableRegions: []auth.GeographicScope{
            {Type: auth.GeoTypeNational, Identifier: "US"},
            {Type: auth.GeoTypeRegional, Identifier: "EU"},
        },
        AuthorizedActions: auth.AuthorizedActions{
            Transactions: []auth.TransactionType{
                auth.TransactionPurchase, auth.TransactionSale,
            },
            Decisions: []auth.DecisionType{
                auth.DecisionFinancial, auth.DecisionInformation,
            },
            NonPhysicalActions: []auth.NonPhysicalAction{
                auth.ActionResearch, auth.ActionSharing,
```

```
            },
        },
    },

    // C. Requirements
    Requirements: auth.Requirements{
        ValidityPeriod: auth.ValidityPeriod{
            StartTime: time.Now(),
            EndTime:   time.Now().Add(90 * 24 * time.Hour),
            TimeWindows: []auth.TimeWindow{
                {Start: "09:00", End: "17:00", Timezone: "EST"},
            },
        },
        PowerLimits: auth.PowerLimits{
            PowerLevels: []auth.PowerLevel{
                {Type: "transaction_value", Limit: 500000.0, Currency: "USD"},
                {Type: "daily_limit", Limit: 1000000.0, Currency: "USD"},
            },
            QuantumResistance: true,
            ExplicitExclusions: []string{"cryptocurrency_trading"},
        },
        JurisdictionLaw: auth.JurisdictionLaw{
            Language:           "English",
            GoverningLaw:       "Delaware_Corporate_Law",
            PlaceOfJurisdiction: "US",
        },
    },
}

// Create AgentAuth request
request := auth.AgentAuthRequest{
    ClientID:     "ai_financial_advisor_v3",
    ResponseType: "code",
    Scope:        []string{"financial_advisory", "asset_management"},
State:        "secure_state_token_2025", # (for demonstration only; NOT production ready)
    PowerType:    "financial_advisory_powers",
    PrincipalID:  "GlobalTech-Corp-2025",
    AIAgentID:    "ai_financial_advisor_v3",
    Jurisdiction: "US",
    PoADefinition: poa,
}

// Authorize with full RFC validation
response, err := service.AuthorizeAgentAuth(context.Background(), request)
if err != nil {
    fmt.Printf("  Authorization failed: %v\n", err)
    return
}
```

```
    fmt.Printf("  Authorization successful!\n")
    fmt.Printf("Authorization Code: %s\n", response.AuthorizationCode[:20]+"...")
    fmt.Printf("Legal Compliance: %v\n", response.LegalCompliance)
    fmt.Printf("Compliance Level: %s\n", response.PoAValidation.ComplianceLevel)
    fmt.Printf("Attestation Status: %s\n", response.PoAValidation.AttestationStatus)
    fmt.Printf("Audit Record: %s\n", response.AuditRecordID)
}
```

This API reference provides complete documentation for the official AgentAuth Community AgentAuth RFC implementation. For additional examples and guides, see the Getting Started Guide and RFC Architecture Documentation.

1.16 Infrastructure & Resilience Utilities

1.16.1 Circuit Breaker

The circuit breaker now uses an options-driven constructor:

```
type Options struct {
    Name             string
    FailureThreshold int
    ResetTimeout     time.Duration
    HalfOpenLimit    int // reserved
}

cb := circuit.NewBreaker(circuit.Options{ Name: "auth-service", FailureThreshold: 5, ResetTimeo
err := cb.Execute(ctx, func() error { return maybeFailingOperation() })
state := cb.GetState() // circuit.StateClosed | StateOpen | StateHalfOpen
```

Deprecated shim retained (will be removed in a future release):

```
// Deprecated: use NewBreaker
legacy := circuit.New("auth-service", 5, 10*time.Second)
```

1.16.2 Rate Limiter (Simplified)

Unified token bucket implementation in `pkg/rate`:

```
type Config struct {
    RequestsPerSecond int
    BurstSize         int
    WindowSize        time.Duration // used for aliasing / demos
}

lim := rate.NewLimiter(rate.Config{RequestsPerSecond: 50, BurstSize: 100, WindowSize: time.Seco
if lim.Allow() { /* proceed */ }
```

```
_ = lim.Wait(ctx) // blocks until allowed
remaining := lim.GetRemainingRequests("client-id") // approximate
```

Aliases for demo compatibility:

```
enh := rate.NewEnhancedConfig(25, time.Second) // returns EnhancedConfig with underlying Confi
tb := rate.NewTokenBucket(enh)                 // wrapper with Allow() error
```

Removed / Deprecated: - Sliding window & distributed public constructors (examples and benchmarks now skipped / removed).

1.16.3 Metrics Collector

Enhanced collector returns aggregated metric map:

```
mc := monitoring.NewMetricsCollector()
mc.IncrementWithLabels("transactions_total", map[string]string{"status":"success"})
mc.GaugeWithLabels("response_time_seconds", 0.123, nil)
metrics := mc.GetAllMetrics() // map[string]monitoring.MetricValue{ Name: {Value, Labels} }
```

Structural change: `GetAllMetrics()` no longer returns per-label series grouped by name — tests should assert presence and basic value thresholds rather than specific label combinations.

1.16.4 Authorization Policy Model Update

New simplified policy struct (plural resource/action containers removed):

```
type Policy struct {
    Subject  string
    Resource string
    Actions  []string
    Effect   string // typically "allow" or "deny"
}

authorizer.AddPolicy(Policy{Subject: "user-1", Resource: "payments", Actions: []string{"read",'
decision := authorizer.Authorize(Request{Subject: "user-1", Resource: "payments", Action: "rea
if decision.Allow { /* authorized */ }
```

1.16.5 Audit Event API Change

Deprecated fluent builder (`audit.NewEntry()...WithActor()...`) replaced by event factory:

```
evt := audit.NewEvent(audit.EventTypeAuthentication, audit.ActionLogin, audit.ResultSuccess)
evt.Subject = "user123"
logger := audit.NewAuditLogger()
_ = logger.Log(ctx, evt)
```

Benchmarks updated accordingly; the old chain methods are removed.

1.16.6 Token Store Changes

Distributed token store helpers (`NewDistributedStore`, `DistributedConfig`) removed from integration tests (no implementation in current module scope). Memory store remains primary:

```
store := token.NewMemoryStore(time.Hour)
// save / get / revoke tokens
```

1.16.7 HTTP Deprecation Headers

Beta HTTP routes now emit the following headers to signal API evolution:

```
X-API-Deprecated: true
X-API-Replacement: /api/v1/beta
```

Clients should plan migration paths accordingly.

Chapter 2

AgentAuth API Versioning & Deprecation Policy

Version: 1.0
Effective Date: November 2025
Last Updated: November 15, 2025

2.1 Table of Contents

2.2 Overview

AgentAuth follows a clear and predictable versioning strategy to ensure stability for integrators while allowing the API to evolve. This document outlines our commitment to backward compatibility and provides guidance on managing API changes.

2.2.1 Key Principles

1. **Stability**: Existing integrations should continue to work without modification
2. **Predictability**: Changes are announced well in advance with clear timelines

3. **Transparency**: All changes are documented with migration guides
4. **Developer-First**: Decisions prioritize developer experience and ease of migration

---

2.3 Versioning Strategy

2.3.1 URL-Based Versioning

AgentAuth uses **URL-based versioning** as the primary versioning mechanism:

`https://api.agentauth.example.com/api/{version}/{endpoint}`

Examples: - `/api/v1/beta/health` - Version 1 (beta) - `/api/v1/aap001/subscriptions` - Version 1 (stable) - `/api/v2/subscriptions` - Version 2 (future)

2.3.2 Version Header Support

Optionally, clients can specify versions using headers:

```
GET /api/health HTTP/1.1
X-API-Version: v1
```

If both URL and header versions are present, the **URL version takes precedence**.

2.3.3 Semantic Versioning for SDKs

Client SDKs follow Semantic Versioning 2.0.0:

- **MAJOR** (1.0.0 → 2.0.0): Breaking changes
- **MINOR** (1.0.0 → 1.1.0): New features, backward compatible
- **PATCH** (1.0.0 → 1.0.1): Bug fixes, backward compatible

2.4 Current Versions

2.4.1 Version 1 (v1) - CURRENT

Status: Beta → Stable (transition planned Q1 2026)
Base Path: `/api/v1`
Release Date: November 2025
End of Life: TBD (minimum 2 years from stable release)

Endpoints: - System: `/api/v1/beta/health`, `/api/v1/beta/info`, `/api/v1/beta/ping` - AAP-001: `/api/v1/aap001/subscriptions/*` - PoA: `/api/v1/beta/poa/*` - Authorization: `/api/v1/beta/authz/*` - PVP: `/api/v1/beta/pvp/*` - Registry: `/api/v1/beta/registry/*` - Policy: `/api/v1/beta/policy/*` - Tokens: `/api/v1/token/*` - Metrics: `/api/v1/beta/metrics/*`

Beta Notice:
Endpoints marked with `/beta/` may receive breaking changes with shorter notice periods (minimum 3 months). Once stable, they will follow the full deprecation policy.

2.5 Deprecation Policy

2.5.1 Deprecation Timeline

When an API version or endpoint is deprecated:

1. **T+0 (Announcement)**: Deprecation is announced via:
   - API response headers (`X-API-Deprecated: true`)
   - Changelog entry
   - Email to registered developers
   - Blog post
   - Documentation update
2. **T+6 Months (Warning Period)**:
   - Deprecation warnings in API responses
   - Migration guide published
   - New version available for testing
3. **T+12 Months (Removal)**:
   - Deprecated version is removed
   - Requests return 410 Gone with migration instructions

2.5.2 Beta Endpoints

Beta endpoints have a **shorter timeline**:

1. **T+0 (Announcement)**: Deprecation announced
2. **T+3 Months (Warning)**: Migration guide available
3. **T+6 Months (Removal)**: Endpoint removed

2.5.3 Deprecation Headers

Deprecated endpoints return the following headers:

```
HTTP/1.1 200 OK
X-API-Deprecated: true
X-API-Deprecated-Date: 2025-11-15T00:00:00Z
X-API-Sunset-Date: 2026-11-15T00:00:00Z
X-API-Migration-Guide: https://docs.agentauth.example.com/migration/v1-to-v2
Link: <https://api.agentauth.example.com/api/v2/subscriptions>; rel="successor-version"
```

2.5.4 Deprecation Response Body

Deprecated endpoints include a warning in the response:

```
{
  "data": { /* response data */ },
  "deprecated": {
    "deprecated_at": "2025-11-15T00:00:00Z",
    "sunset_at": "2026-11-15T00:00:00Z",
    "message": "This endpoint is deprecated and will be removed on 2026-11-15",
    "migration_guide": "https://docs.agentauth.example.com/migration/v1-to-v2",
    "successor": "/api/v2/subscriptions"
```

```
  }
}
```

2.6 Breaking Changes

2.6.1 What Constitutes a Breaking Change?

Breaking changes include:

1. **Removing endpoints or fields** from responses
2. **Renaming fields** in requests or responses
3. **Changing field types** (string → number, etc.)
4. **Changing authentication mechanisms**
5. **Removing or changing error codes**
6. **Changing HTTP methods** (POST → PUT)
7. **Making optional fields required**
8. **Changing URL structure**

2.6.2 What is NOT a Breaking Change?

Non-breaking changes include:

1. **Adding new endpoints**
2. **Adding new optional fields** to requests
3. **Adding new fields** to responses
4. **Adding new error codes** (clients should handle unknown codes gracefully)
5. **Changing internal implementation** without API changes
6. **Improving performance**
7. **Bug fixes** that restore documented behavior

2.6.3 Handling Breaking Changes

Breaking changes are **only introduced in major versions**:

- v1 → v2: Breaking changes allowed
- v1.0 → v1.1: No breaking changes
- v1.0.0 → v1.0.1 (SDK): No breaking changes

2.7 Migration Process

2.7.1 Step-by-Step Migration Guide

When migrating from one version to another:

2.7.1.1 Step 1: Review the Changelog

```
# Check the API changelog
curl https://api.agentauth.example.com/api/changelog
```

2.7.1.2 Step 2: Test Against New Version

```
# Use the new version in a test environment
curl https://api-staging.agentauth.example.com/api/v2/subscriptions
```

2.7.1.3 Step 3: Update Your Code

Before (v1):

```
const response = await fetch('/api/v1/beta/poa', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
  body: JSON.stringify({ grantor, grantee, scope })
});
```

After (v2):

```
const response = await fetch('/api/v2/authorizations', {
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
  body: JSON.stringify({
    principal: grantor,
    delegate: grantee,
    permissions: scope
  })
});
```

2.7.1.4 Step 4: Update SDK Version

```
# JavaScript/TypeScript
npm install @agentauth/client@2.0.0

# Python
pip install agentauth-client==2.0.0
```

2.7.1.5 Step 5: Deploy and Monitor

- Deploy to staging first
- Run integration tests
- Monitor error rates
- Gradually roll out to production

2.8 Backward Compatibility

2.8.1 Compatibility Guarantees

Within a major version (e.g., v1):

Guaranteed Compatible: - Existing endpoints will not be removed - Existing fields will not be renamed - Existing field types will not change - Authentication mechanisms remain stable

May Be Added (Non-Breaking): - New optional request fields - New response fields - New endpoints - New error codes

Not Guaranteed: - Exact error messages (use error codes instead) - Response field ordering - Undocumented behavior

2.8.2 Forward Compatibility

Design your clients to be forward-compatible:

1. **Ignore unknown fields** in responses
2. **Handle new error codes** gracefully (treat as generic errors)
3. **Don't rely on field ordering** in JSON responses
4. **Parse timestamps** as ISO 8601, not specific formats
5. **Follow redirects** (3xx responses)

Example: Forward-Compatible Client

```
def parse_subscription(data):
    # Extract only the fields we know about
    return {
        'id': data['id'],
        'status': data.get('status', 'unknown'),
        'client_id': data.get('client_id')
        # Unknown fields are ignored automatically
    }
```

2.9 API Lifecycle

2.9.1 Lifecycle Stages

`Experimental → Beta → Stable → Deprecated → Retired`

2.9.1.1 1. Experimental (Optional)

- **Duration**: 1-3 months
- **Path**: `/api/experimental/*`
- **Guarantees**: None - may change at any time
- **Notice**: Not for production use

2.9.1.2 2. Beta

- **Duration**: 3-12 months
- **Path**: `/api/v1/beta/*`
- **Guarantees**: 3-month deprecation notice
- **Notice**: Use cautiously in production

2.9.1.3 3. Stable

- **Duration**: 2+ years

- **Path**: `/api/v1/*` (no `/beta/`)
- **Guarantees**: 12-month deprecation notice
- **Notice**: Production-ready

2.9.1.4 4. Deprecated

- **Duration**: 6-12 months
- **Headers**: `X-API-Deprecated: true`
- **Guarantees**: Continues to work during deprecation period
- **Notice**: Migrate to new version

2.9.1.5 5. Retired

- **Status**: 410 Gone
- **Response**: Migration instructions
- **Notice**: No longer available

2.10 Version Support Timeline

2.10.1 Active Support

- **Full Support**: Bug fixes, security patches, feature updates
- **Duration**: Current major version + previous major version

2.10.2 Security Support

- **Security Patches Only**: Critical security vulnerabilities
- **Duration**: Previous major version only (1 year after new major release)

2.10.3 End of Life

- **No Support**: No updates or patches
- **Status**: Retired versions return 410 Gone

2.10.4 Example Timeline

```
v1 Released:        Nov 2025           Full Support
v2 Released:                  Nov 2026        Full Support
v1 Deprecated:                Nov 2026     Security Only
v1 Retired:                               Nov 2027    EOL
v3 Released:                                      Nov 2027
v2 Deprecated:                                    Nov 2027
v2 Retired:                                                    Nov 2028    EOL
```

2.11 Best Practices

2.11.1 For API Clients

1. **Always specify version** in URLs (don't rely on defaults)
2. **Check deprecation headers** in responses
3. **Subscribe to changelog** notifications
4. **Test against new versions** early
5. **Use official SDKs** for automatic version handling
6. **Implement graceful degradation** for unknown fields
7. **Monitor API status** page for announcements

2.11.2 For API Developers

1. **Document all changes** in CHANGELOG.md
2. **Announce deprecations** 6+ months in advance
3. **Provide migration guides** with code examples
4. **Version SDKs** according to semver
5. **Maintain OpenAPI specs** for each version
6. **Run breaking change detection** in CI/CD
7. **Support previous version** for 12 months minimum

2.12 Migration Guides

2.12.1 v1-beta → v1-stable (Planned Q1 2026)

Changes: - Remove `/beta/` from stable endpoints - Finalize request/response schemas - Lock in field names and types

Action Required: - Update URLs: `/api/v1/beta/poa` → `/api/v1/poa` - Test integration with stable endpoints

Migration Timeline: - **Dec 2025**: v1-stable available for testing - **Feb 2026**: v1-beta deprecated (6-month notice) - **Aug 2026**: v1-beta retired

---

2.13 Changelog Access

2.13.1 API Changelog Endpoint

```
# Get changelog for current version
curl https://api.agentauth.example.com/api/changelog

# Get changelog for specific version
curl https://api.agentauth.example.com/api/changelog?version=v1
```

2.13.2 GitHub Releases

All versions are tagged and released on GitHub:

https://github.com/mauriciomferz/AgentAuth/releases

2.13.3 RSS Feed

Subscribe to changelog updates:

https://api.agentauth.example.com/api/changelog.rss

2.14 Support and Contact

2.14.1 Questions About Versioning?

- **Email**: api-support@agentauth.example.com
- **GitHub Discussions**: https://github.com/mauriciomferz/AgentAuth/discussions
- **Slack**: #agentauth-api-versioning

2.14.2 Report Issues

- **GitHub Issues**: https://github.com/mauriciomferz/AgentAuth/issues
- **Security Issues**: security@agentauth.example.com

2.15 References

- Semantic Versioning
- RFC 8594 - Sunset HTTP Header
- API Evolution Best Practices

Last Updated: November 15, 2025
Next Review: February 2026